Bryan Cortes

Professor Yanilda Peralta Ramos

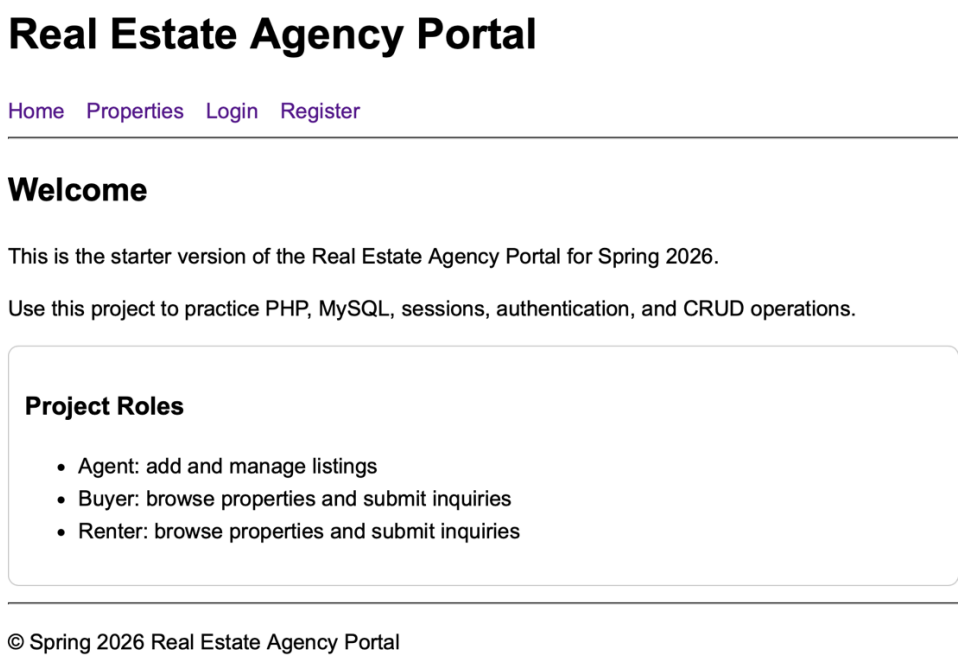
CIS 344

May 6, 2026

Real Estate Agency Portal Report

Project Purpose

The purpose of this project was to build a simple but realistic Real Estate Agency Portal where three different user types (agents, buyers, and renters) can interact with property listings. Agents manage listings, while buyers and renters browse properties and submit inquiries. This project helped me practice core PHP web development skills, including working with forms, connecting to a MySQL database, and building role-based behavior.



Database Design

The database is named real_estate_portal_db and includes five main tables: Users, Properties, Inquiries, Transactions, and Favorites. The Users table stores login information and a userType field (agent, buyer, renter) to support role-based access. The Properties table stores listing details (title, type, address, city, price, status) and includes an agentId foreign key referencing Users(userId). The Inquiries table connects users to properties with a message and timestamp, while the Transactions table stores completed sales or rentals, and the Favorites table allows buyers/renters to save listings. Foreign keys enforce relationships, so inquiries, favorites, and transactions always connect to valid users and properties. Also, PropertyListingViev combines property information with the agent's name to display listings cleanly.

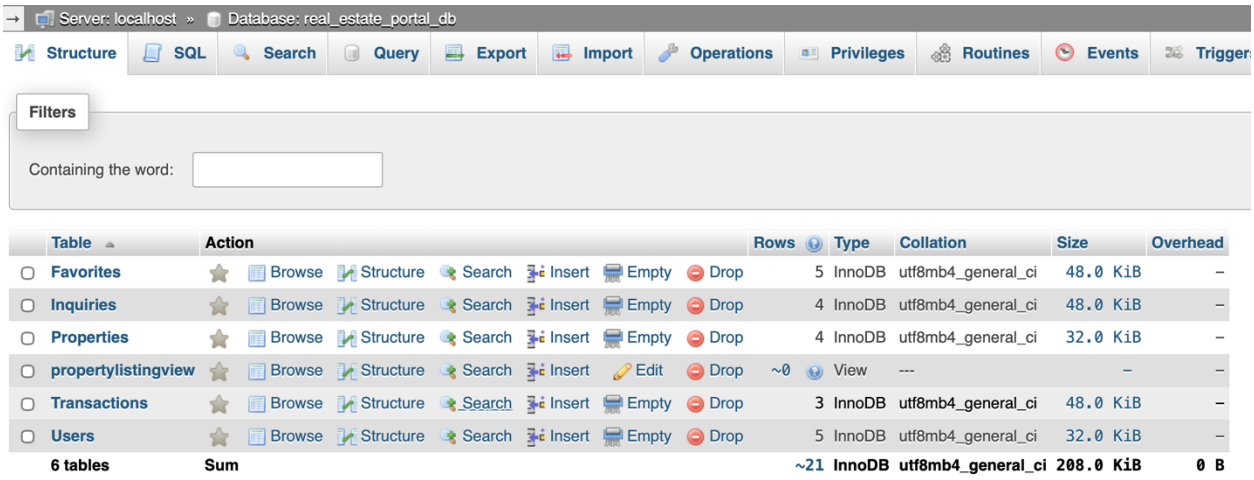
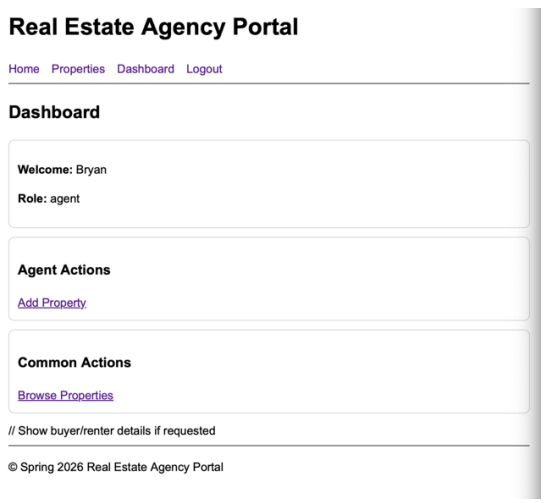


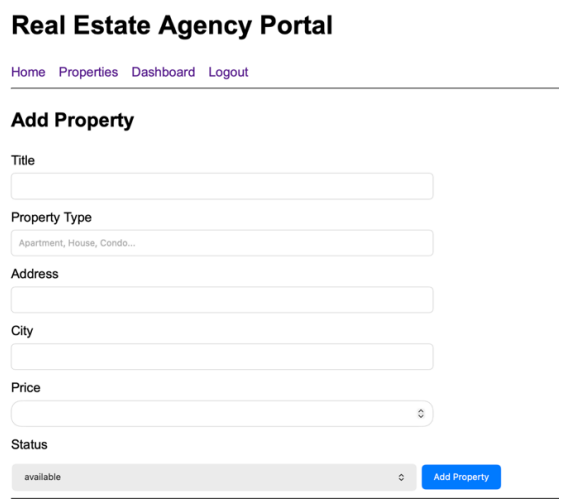
Table	Action	Rows	Type	Collation	Size	Overhead
Favorites	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	48.0 KiB	–
Inquiries	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	48.0 KiB	–
Properties	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	32.0 KiB	–
propertylistingview	Browse Structure Search Insert Edit Drop	~0	View	---	–	–
Transactions	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	48.0 KiB	–
Users	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	32.0 KiB	–
6 tables	Sum	~21	InnoDB	utf8mb4_general_ci	208.0 KiB	0 B

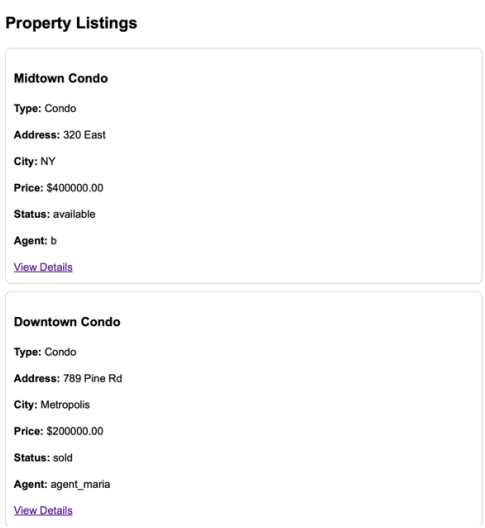
PHP Functionality

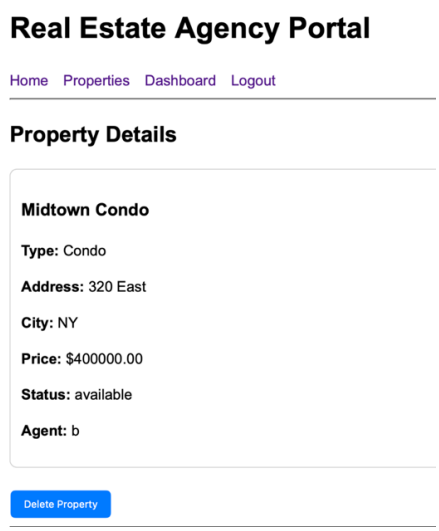
The platform is built using PHP pages and forms connected directly to the database. Key features include browsing listings (properties.php), viewing a single listing (property_details.php), adding properties (add_property.php), submitting inquiries (submit_inquiry.php), and showing a user dashboard (dashboard.php). PHP handles form submissions using POST requests, then runs SQL queries to insert or retrieve records. For example, adding a listing inserts a row into Properties, and submitting an inquiry inserts a row into Inquiries.

Agent:









Buyer:

Real Estate Agency Portal

Home Properties Dashboard Logout

Dashboard

Welcome: Cortes

Role: buyer

Common Actions

Browse Properties

// Show buyer/renter details if requested

Buyer/Renter Actions

View Buyer/Renter Details

© Spring 2026 Real Estate Agency Portal

Real Estate Agency Portal

Home Properties Dashboard Logout

Property Details

Property saved to favorites!

Downtown Condo

Type: Condo

Address: 789 Pine Rd

City: Metropolis

Price: $200000.00

Status: sold

Agent: agent_maria

Save to Favorites Submit Inquiry

Buyer/Renter Actions

View Buyer/Renter Details

Your Info

Username: Cortes

Contact Info: cortes@gmail.com

Role: buyer

Your Favorite Listings

- Downtown Condo (Saved: 2026-05-07 16:46:56)

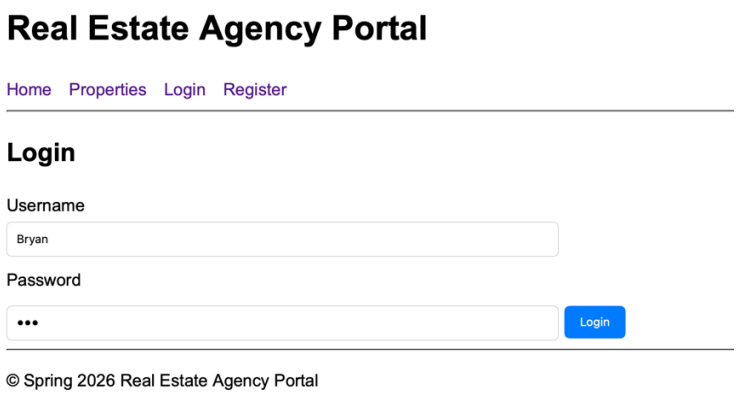
Your Inquiries

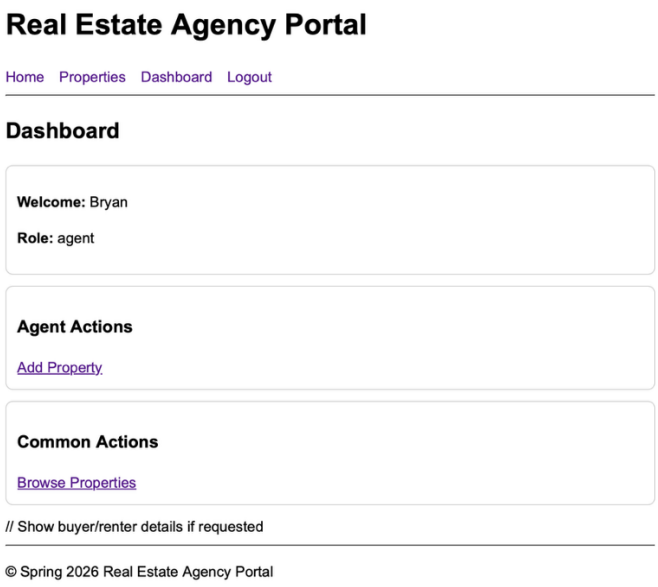
- On **Downtown Condo**: "Love this place" (2026-05-07 16:47:05)

Login and Session Handling

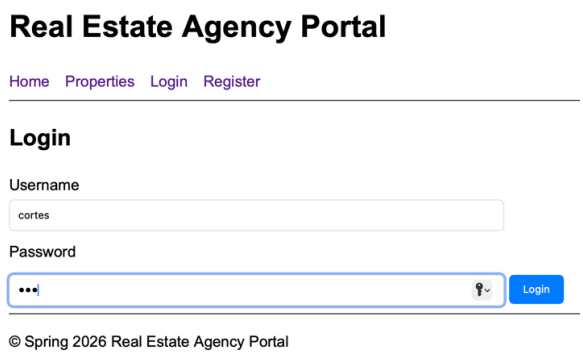
The login system is implemented using password_hash() for secure password storage and password_verify() for checking user passwords at login. After login, PHP sessions store the user's id and role. Logout clears the session and returns the user to a safe page. Role-based access control is enforced by checking userType. Agents are allowed to add and manage listings, while buyers and renters can browse listings, save favorites, and submit inquiries. The dashboard also changes based on the user role, so each user only sees actions that match their permissions

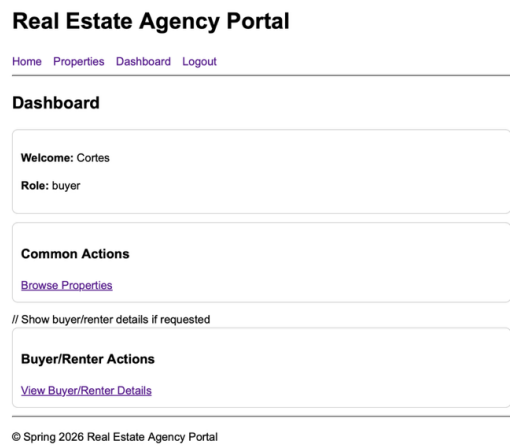
Agent:



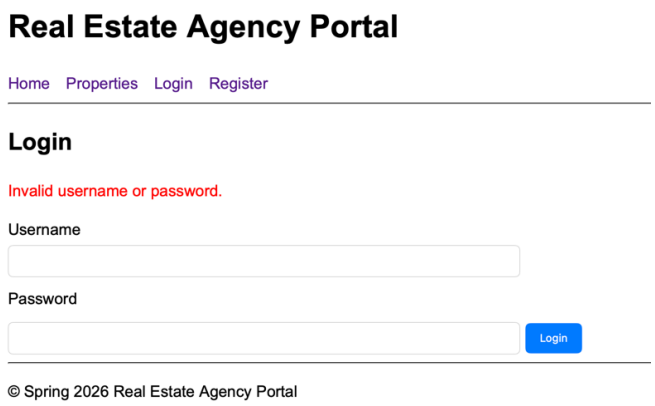


Buyer:





Invalid username or password:



Challenges and Solutions

One challenge was making sure database imports worked correctly in my local environment (XAMPP/MariaDB) and had to update them. I also had to debug small issues like comparing strings that included newline characters from input, which I solved using trim()before comparisons. Another challenge was making sure the correct role behavior displayed across different pages, which I solved by consistently checking session variables and validating access on every restricted page.